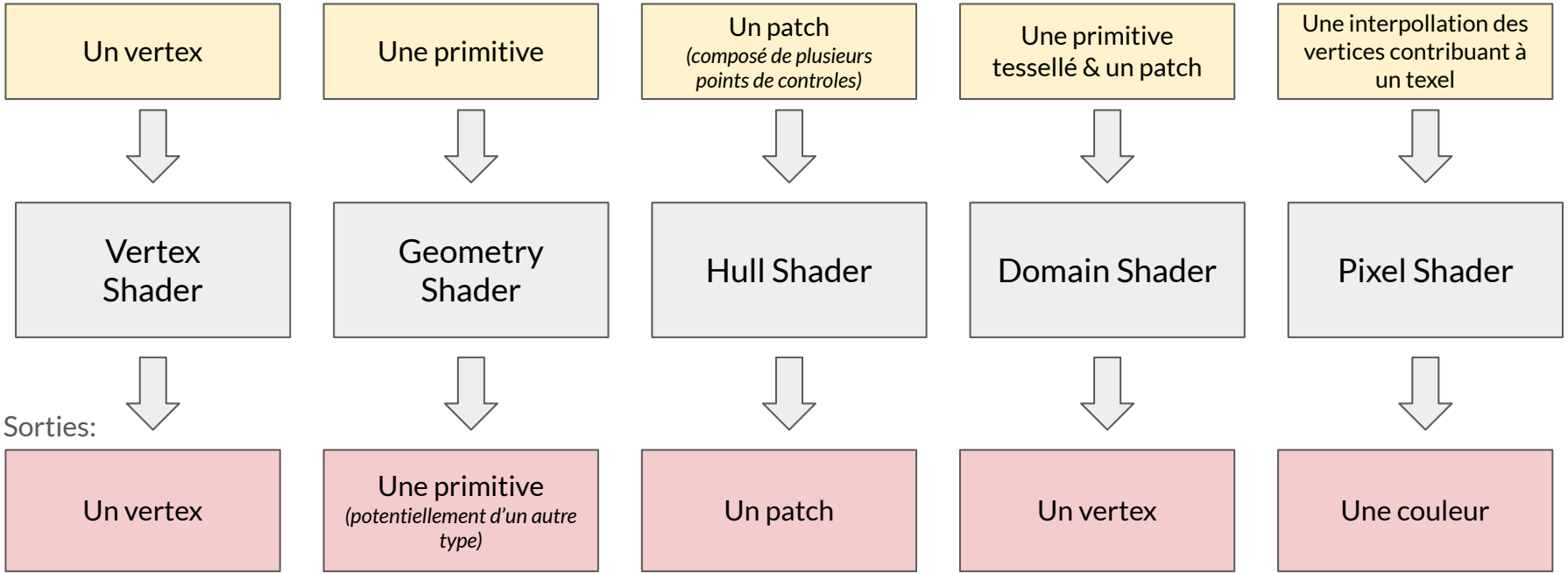
Compute Shader

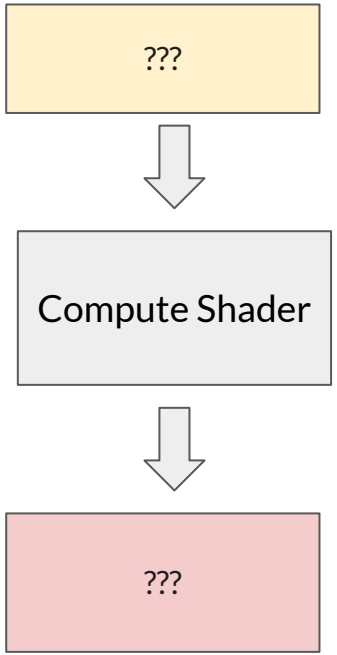
Entrée/sortie des shaders

Entrées:

+ des constantes, des textures, des samplers, etc



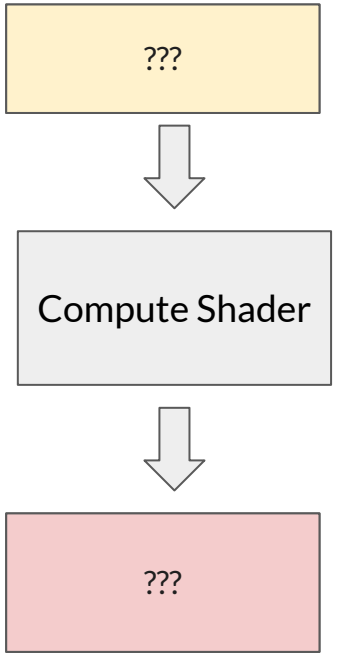
Compute Shader - Entrée/sortie



Mais du coup que prends un compute shader en entrée?

Et surtout, qu’est ce qu’il nous renvoie en sortie?

Compute Shader - Entrée/sortie



Il prends un entrée un index et nous renvoie…

Rien du tout

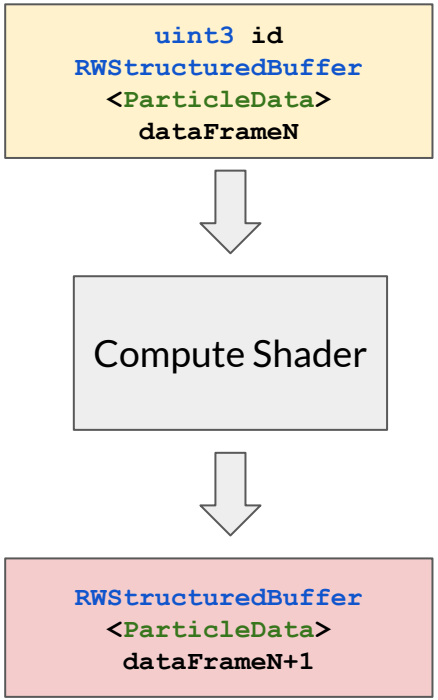
L’interet de ce type de shader réside dans les types de ressources utilisable!

Nouveau type de **ressources**

```
// Nous pouvons préciser RW avant un type pour préciser que c'est une
ressource accesible en écriture! Attention cependant il faut les
mettre dans un registre de type u (pour Unordered Access)
RWTexture2D<float4> writableTexture : register(u0);

// Nous avons aussi accès aux buffers "structuré" qui sont equivalent
à des std::array<TData>, mais utilisable en shader. Ils possedent des
opération atomique simplifiant le parralelisme
struct Data {
    float4 lorem_ipsum;
};
RWStructuredBuffer<Data> writableBuffer : register(u1);
AppendStructuredBuffer<Data> appendableBuffer : register(u2);
ConsumeStructuredBuffer<Data> consumableBuffer : register(u3);
StructuredBuffer<Data> readonlyBuffer : register(t0);
```

Compute Shader - Pourquoi?



Prenons un exemple concret:

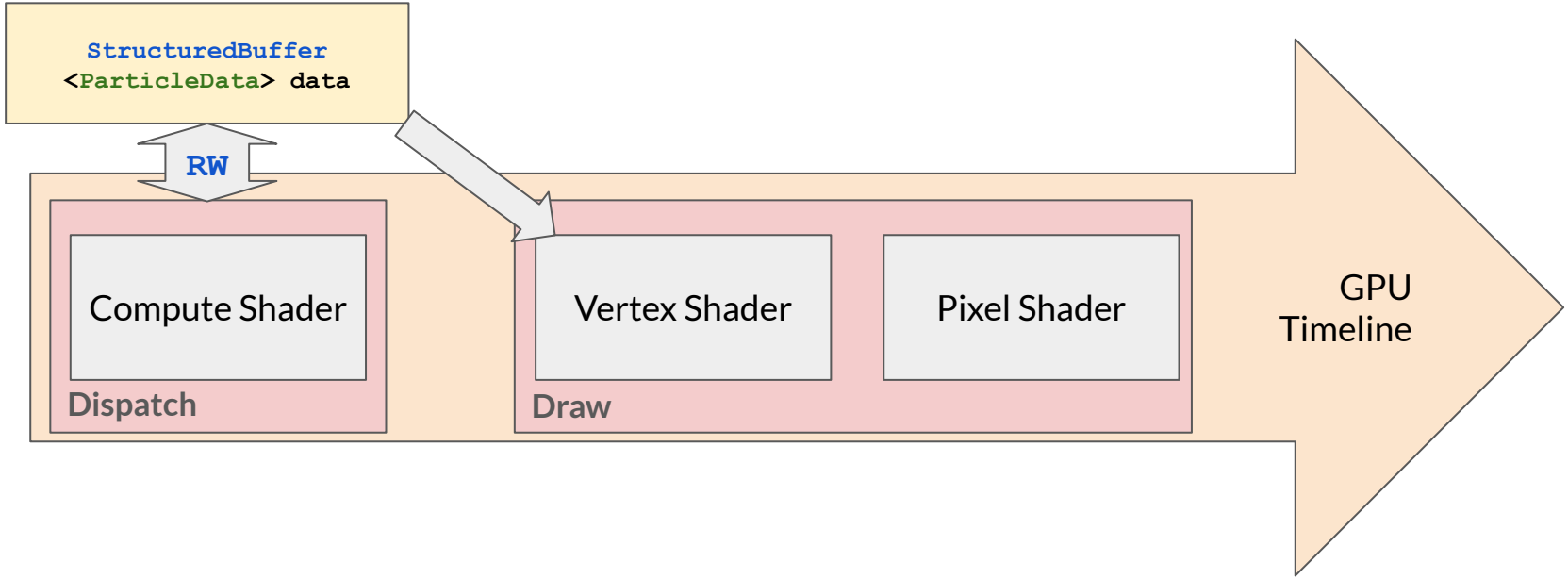
- Nous souhaitons simuler un grand nombre de particules
- Nous ne voulons pas itéré sur chaque particule sur le CPU car notre jeu hypothetique est déja bottlenecked dessus.

C'est un cas d'utilisation typique de compute shader!

- Nous designerons un compute shader qui fera avancer la simulation de *x* millisecondes
- Notre CPU déclarera la ressource graphique contenant nos particules puis elle restera dans les mains du GPU pour le reste de la vie de l'application.
- Le CPU n'aura plus qu'à lancer les commandes GPU pour planifier la simulation et l'affichage

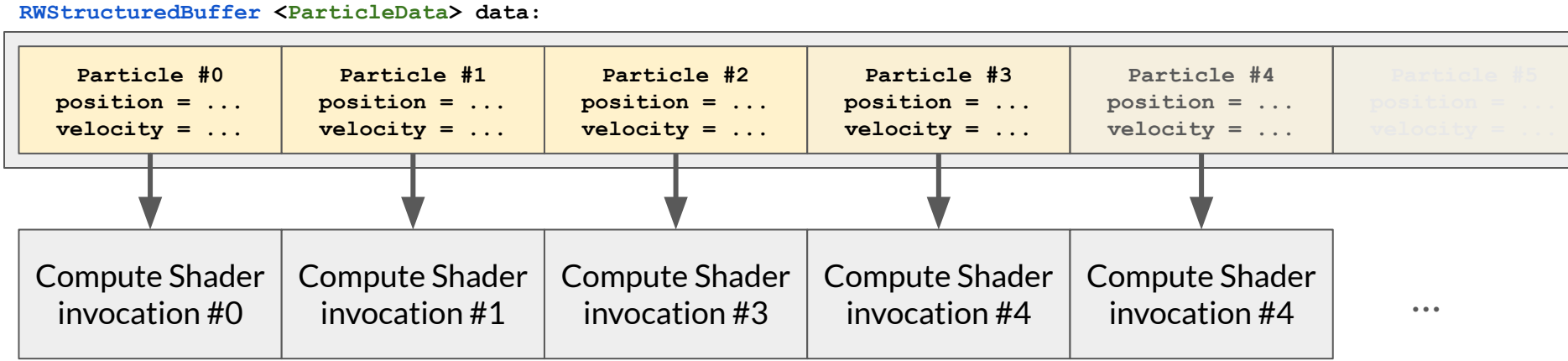
Compute Shader - Exemple de timeline

Utilisation de la ressource:



Compute Shader - Comment ?

Comme vu précédemment, le GPU est hautement parallélisé. Plusieurs invocation de shaders vont avoir lieu en même temps ! *(c'est tout l'intérêt d'ailleurs)*
Pour éviter les cas de race conditions, il faudra faire attention à ce que chaque shaders manipule les données de façon prévisible. C'est la que notre index d'entrée rentre en jeu!



Compute Shader - HLSL (Unity)

```
#pragma kernel CSMain // Unity-specific, précise le point d'entrée

struct ParticleData {
    float3 position;
    float3 velocity;
};
RWStructuredBuffer<ParticleData> particleBuffer; // pas de
register, Unity gère ça pour nous

[numthreads(64, 1, 1)]// combien d'invocation en parallèle, 64 en x ici
void CSMain (uint3 id : SV_DispatchThreadID) {
    // id.x sera compris entre 0 et 63
}
```

Compute Shader - Limitation

Le `[numthreads(64, 1, 1)]` défini en début de fonction permet d'informer le GPU de combien de tâche en parallèles doivent être lancés.
On peut définir un nombre de tâche sur trois dimensions, ça change rien d'un point de vue GPU mais cela rends nos index plus pratique si l'on souhaite manipuler des textures!

On appelle un ensemble de threads effectué en parallèle par le GPU un "**thread group**".
Le GPU invoquera un **thread**, une invocation du compute shader, pour chaque paires de coordonnées dans ce **thread group**.

Le nombre de thread dans un thread group ne peut pas dépasser **1024** *(et peut être plus bas encore selon le backend utilisé, la version de DirectX, le hardware, etc)*.

Compute Shader - Limitation

Exemple: vous souhaitez manipuler une RWTexture2D qui fait 32x32 pixels.

- Vous pouvez définir 32 threads en x et 32 threads en y `[numthreads(32, 32, 1)]`
- Total de thread 32x32=**1024** threads totaux. Nous sommes à notre limite!
- Cela permet de manipuler de façon sûr 32x32 pixels de notre texture ***sans race condition***. Chaque thread manipule un pixel différent.

Cependant nous avons un soucis maintenant, 32x32 pixels maximum c'est pas beaucoup!

Compute Shader - Dispatch

Comment faire pour avoir plus que 1024 threads?

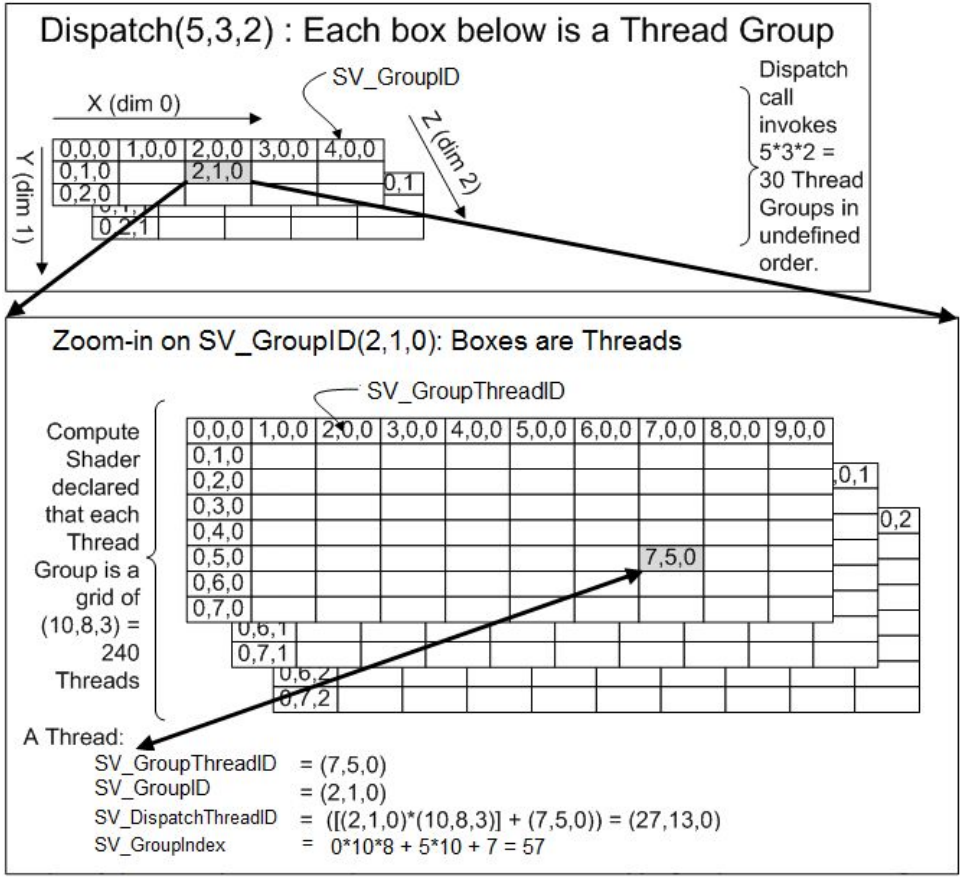
En répétant l'opération plusieurs fois tout simplement!

Entre en jeu notre nouvelle commande GPU:

```
ID3D11DeviceContext::Dispatch(
    (UINT) ThreadGroupCountX,
    (UINT) ThreadGroupCountY,
    (UINT) ThreadGroupCountZ);
```

De la même manière que `[numthreads(x, y, z)]` définissait le nombre de threads dans un group, `Dispatch` défini le nombre de thread group à planifier au total!

Compute Shader



Compute Shader - System-Value Semantics

Nous avons plusieurs type d'index disponible en HLSL avec lesquels jouer:

- `SV_GroupThreadID` qui contient notre coordonnée au sein d'un thread group (compris entre 0,0,0 et la valeur défini dans `numthreads` (exclu))
- `SV_GroupIndex` pareil que `SV_GroupThreadID` mais "aplati" sur une dimension
- `SV_GroupID` coordonnée du Dispatch dans lequel on est
- `SV_DispatchThreadID` le plus utile de tous, coordonnées global de notre invocation, est égale à `SV_GroupID * numthreads + SV_GroupThreadID`